

GESTIONE DEI PERMESSI

Lettura, Scrittura ed Esecuzione in Linux

EPICODE : Cybersecurity & Ethical Hacking
Lorenzo Ventanni

1. Introduzione

In ambiente Linux, la gestione dei permessi rappresenta uno degli strumenti fondamentali per il controllo degli accessi alle risorse del sistema. Ogni file e ogni directory sono associati a tre categorie di utenti, ciascuna con le proprie autorizzazioni di lettura, scrittura ed esecuzione.

I permessi vengono assegnati al proprietario del file (user), al gruppo di appartenenza (group) e agli altri utenti (others). Ogni categoria può ricevere in modo indipendente le seguenti autorizzazioni:

- r (read): permesso di lettura del contenuto
- w (write): permesso di scrittura o modifica
- x (execute): permesso di esecuzione (per i file) o di accesso (per le directory)

In questa esercitazione ho applicato concretamente tali concetti su una directory e su un file di testo, operando sia come utente root sia come utente standard kali, per verificare il comportamento del sistema al variare dei permessi configurati.

2. Obiettivi

Gli obiettivi principali dell'esercitazione sono stati:

- Creare una directory e un file su Kali Linux
- Verificare i permessi iniziali assegnati automaticamente dal sistema con ls -l
- Modificare i permessi tramite il comando chmod
- Testare gli effetti delle modifiche tentando operazioni di lettura e accesso da un utente diverso
- Documentare i risultati ottenuti e motivare le scelte operate

3. Ambiente e Strumenti

3.1 Ambiente di laboratorio

- Sistema operativo: Kali Linux 2026.1 (guest su Oracle VirtualBox)
- Utente principale: kali (standard) con escalation tramite sudo su
- Percorso di lavoro: /home/kali/Desktop/test/test_permessi

3.2 Strumenti e comandi utilizzati

Comando	Descrizione
<code>mkdir</code>	Crea una nuova directory nel percorso specificato
<code>echo ... > file</code>	Crea un file di testo e vi scrive il contenuto indicato
<code>ls -l</code>	Elenca file e directory con i dettagli dei permessi (formato lungo)
<code>chmod</code>	Modifica i permessi di file e directory (modalità ottale o simbolica)
<code>sudo su</code>	Esegue una shell come superutente (root) per operazioni privilegiate
<code>cat</code>	Legge e stampa il contenuto di un file nel terminale
<code>cd</code>	Naviga tra le directory del filesystem

4. Svolgimento

PUNTO 1 Creazione della Directory e Verifica dei Permessi Iniziali

Come prima operazione, ho aperto un terminale su Kali Linux e acquisito i privilegi di root tramite il comando `sudo su`. Successivamente ho creato la struttura di directory necessaria per l'esercitazione:

```
$ sudo su
# mkdir test
# cd test
# mkdir test_permessi
# ls -l
```

Il comando `ls -l` ha mostrato la directory appena creata con i seguenti permessi:

```
drwxr-xr-x  2 root root 4096 Jun 9 09:00 test_permessi
```

Questo output indica che il sistema ha assegnato automaticamente il valore di default 755 alla directory.

Il proprietario root dispone di lettura, scrittura ed esecuzione (rwx), mentre gruppo e altri utenti hanno solo lettura ed esecuzione (r-x), senza la possibilità di creare o cancellare file all'interno.

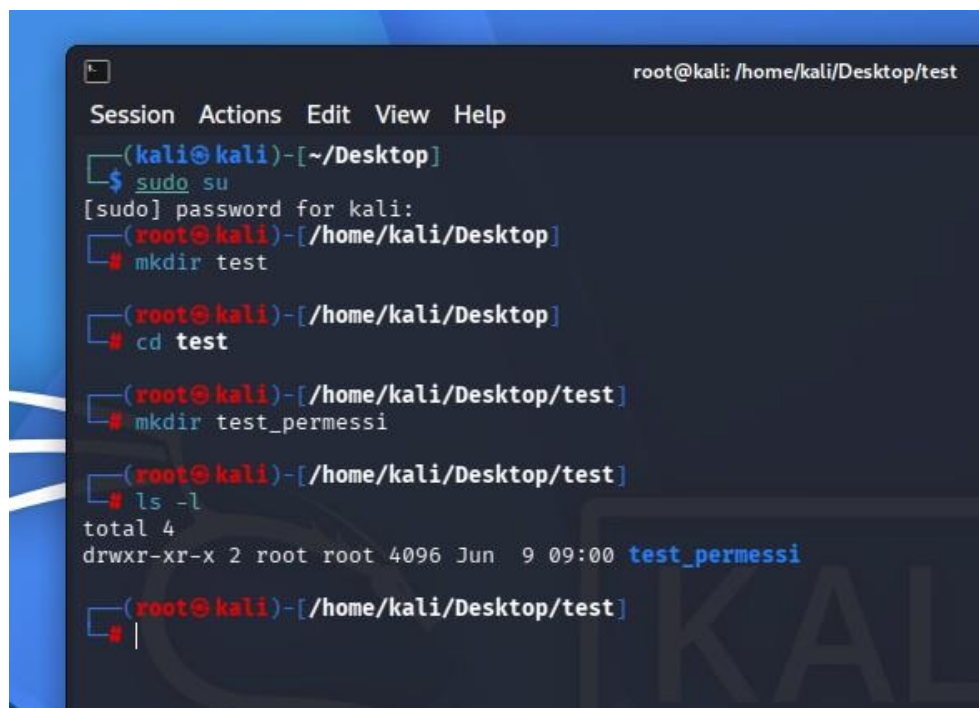


Figura 1 - Punto 1 e 2: creazione della directory `test_permessi` e verifica dei permessi iniziali con `ls -l`

PUNTO 1b Creazione del File e Permessi Iniziali

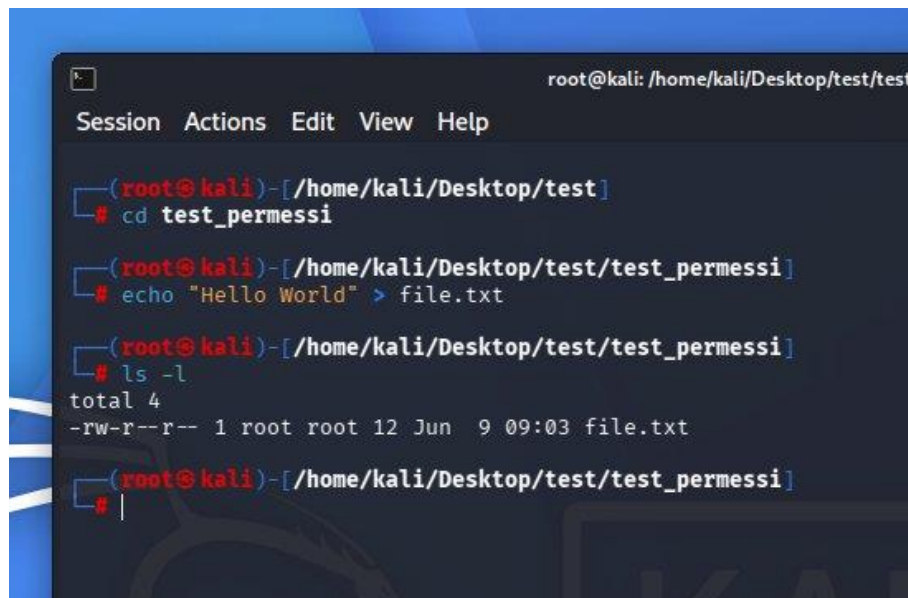
Una volta entrato nella directory `test_permessi`, ho creato un file di testo denominato `file.txt` contenente la stringa "Hello World" tramite il reindirizzamento dell'output:

```
# cd test_permessi
# echo "Hello World" > file.txt
# ls -l
```

L'output del comando `ls -l` ha restituito:

```
-rw-r--r--  1 root root 12 Jun 9 09:03 file.txt
```

Il file è stato creato con permessi 644: il proprietario root ha lettura e scrittura (rw-), mentre gruppo e altri hanno solo lettura (r--). Nessuno dispone dei permessi di esecuzione, il che è corretto per un semplice file di testo.



```
root@kali: /home/kali/Desktop/test/test
Session Actions Edit View Help

(root@kali)-[/home/kali/Desktop/test]
# cd test_permessi

(root@kali)-[/home/kali/Desktop/test/test_permessi]
# echo "Hello World" > file.txt

(root@kali)-[/home/kali/Desktop/test/test_permessi]
# ls -l
total 4
-rw-r--r-- 1 root root 12 Jun  9 09:03 file.txt

(root@kali)-[/home/kali/Desktop/test/test_permessi]
#
```

Figura 2 - Creazione di file.txt nella directory test_permessi e verifica dei permessi iniziali

PUNTO 3 Modifica dei Permessi

Con il file creato, ho proceduto alla modifica dei permessi per osservare gli effetti pratici sugli accessi. La prima operazione è stata impostare permessi esclusivi per il proprietario su file.txt:

```
# chmod 700 file.txt
# ls -l
```

Il risultato è stato:

```
-rwx----- 1 root root 12 Jun 9 09:03 file.txt
```

Il valore 700 assegna tutti i permessi (rwx) solo al proprietario root, negando completamente l'accesso a gruppo e altri. Successivamente ho anche ampliato i permessi della directory test_permessi a 777 per consentire all'utente kali di accedervi. Si è verificato che l'utente kali non poteva modificare i permessi della directory senza privilegi di root, confermando il corretto funzionamento del modello di controllo degli accessi:

```
$ cd test_permessi
cd: permission denied: test_permessi
$ chmod 777 test_permessi
chmod: changing permissions of 'test_permessi': Operation not permitted
$ sudo su
# chmod 777 test_permessi
# exit
$ ls -l
drwxrwxrwx 2 root root 4096 Jun 9 09:03 test_permessi
```

```
(root@kali)-[/home/kali/Desktop/test/test_permessi]
# chmod 700 file.txt

(root@kali)-[/home/kali/Desktop/test/test_permessi]
# ls -l
total 4
-rwx----- 1 root root 12 Jun  9 09:03 file.txt

(root@kali)-[/home/kali/Desktop/test/test_permessi]
# exit

(kali@kali)-[~/Desktop/test]
$ cd test_permessi
cd: permission denied: test_permessi

(kali@kali)-[~/Desktop/test]
$ chmod 777 test_permessi
chmod: changing permissions of 'test_permessi': Operation not permitted

(kali@kali)-[~/Desktop/test]
$ sudo su
(root@kali)-[/home/kali/Desktop/test]
# chmod 777 test_permessi

(root@kali)-[/home/kali/Desktop/test]
# exit

(kali@kali)-[~/Desktop/test]
$ ls -l
total 4
drwxrwxrwx 2 root root 4096 Jun  9 09:03 test_permessi
```

Figura 3 - Punto 3: modifica dei permessi con chmod 700 su file.txt e chmod 777 su test_permessi

Nota: L'impossibilit  da parte dell'utente kali di eseguire chmod sulla directory di propriet  di root dimostra il principio del privilegio minimo: solo il proprietario del file (root) pu  alterarne i permessi.

PUNTO 4 Test dei Permessi

Con la directory test_permessi ora accessibile a tutti (777), abbiamo eseguito una serie di test sistematici per verificare il comportamento del sistema al variare dei permessi di file.txt. Di seguito il riepilogo delle configurazioni testate:

chmod	Permessi	cat (kali)	Note
444	-r--r--r--	Consentito	Tutti leggono, nessuno scrive
744	-rwxr--r--	Consentito	Proprietario: tutti i diritti; altri: solo lettura
700	-rwx-----	Negato	Solo il proprietario accede; kali ottiene "Permission denied"

Il test con chmod 444 ha assegnato permessi di sola lettura a tutti gli utenti. L'utente kali ha potuto leggere il file con cat, ottenendo come output "Hello World".

Risultato: chmod 444 - cat file.txt come utente kali: "Hello World" (lettura riuscita)

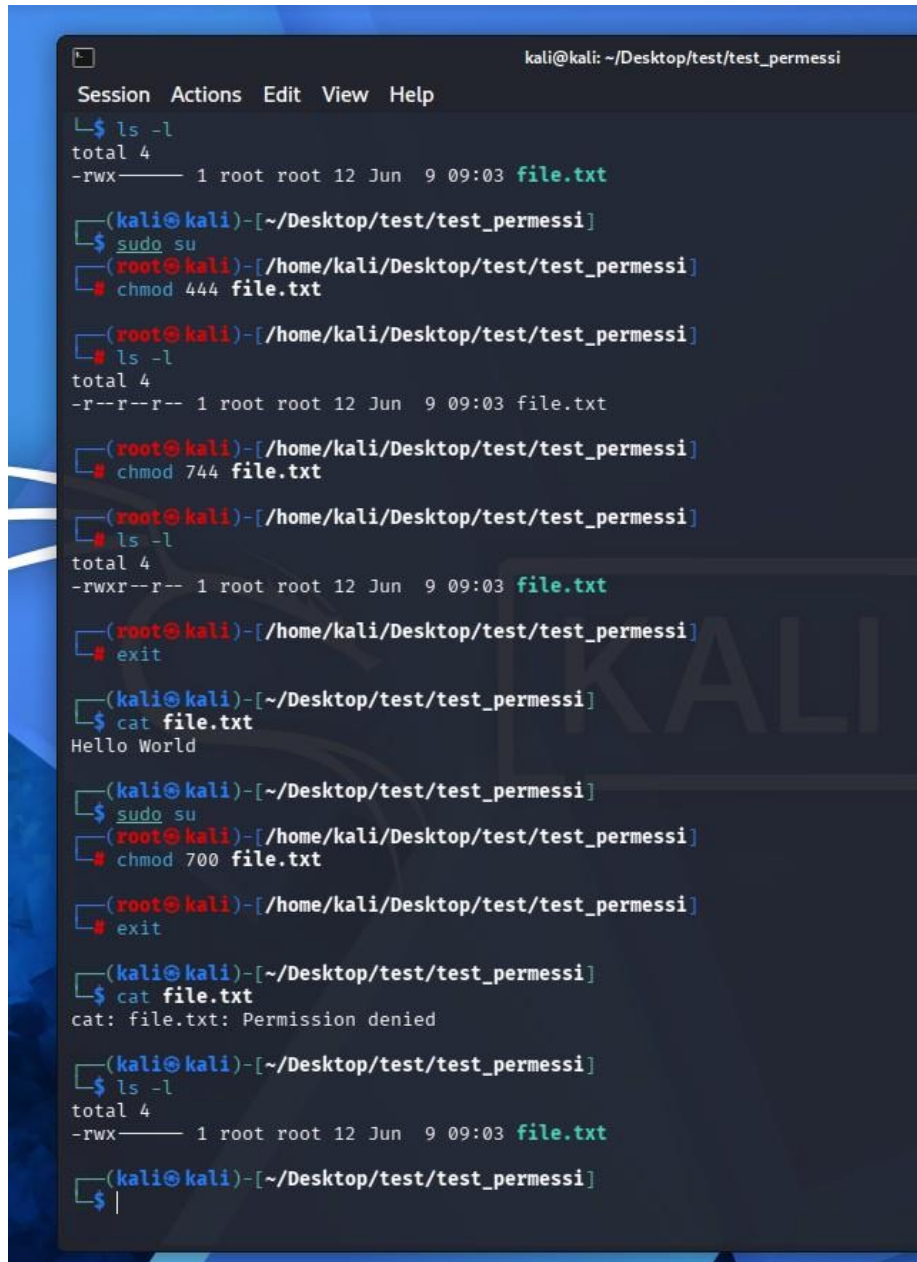
Con chmod 744 il proprietario root dispone di tutti i permessi, mentre gli altri utenti hanno solo lettura. Anche in questo caso kali ha potuto leggere il file senza problemi.

Risultato: chmod 744 - cat file.txt come utente kali: "Hello World" (lettura riuscita)

Infine, ripristinando chmod 700, tutti i permessi sono stati revocati ai non-proprietari. Il tentativo di kali di leggere il file ha prodotto un errore esplicito da parte del kernel:

```
$ cat file.txt  
cat: file.txt: Permission denied
```

Accesso negato: chmod 700 - cat file.txt come utente kali: "Permission denied" (accesso correttamente negato)



```
kali@kali: ~/Desktop/test/test_permessi  
Session Actions Edit View Help  
$ ls -l  
total 4  
-rwx----- 1 root root 12 Jun  9 09:03 file.txt  
  
$ sudo su  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# chmod 444 file.txt  
  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# ls -l  
total 4  
-r--r--r-- 1 root root 12 Jun  9 09:03 file.txt  
  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# chmod 744 file.txt  
  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# ls -l  
total 4  
-rwxr--r-- 1 root root 12 Jun  9 09:03 file.txt  
  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# exit  
  
(kali@kali)~[/Desktop/test/test_permessi]  
$ cat file.txt  
Hello World  
  
(kali@kali)~[/Desktop/test/test_permessi]  
$ sudo su  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# chmod 700 file.txt  
  
(root@kali)~[/home/kali/Desktop/test/test_permessi]  
# exit  
  
(kali@kali)~[/Desktop/test/test_permessi]  
$ cat file.txt  
cat: file.txt: Permission denied  
  
(kali@kali)~[/Desktop/test/test_permessi]  
$ ls -l  
total 4  
-rwx----- 1 root root 12 Jun  9 09:03 file.txt  
  
(kali@kali)~[/Desktop/test/test_permessi]  
$
```

Figura 4 - Punto 4: test dei permessi con chmod 444, 744 e 700 su file.txt; lettura consentita e negata a seconda della configurazione

5. Relazione

5.1 Motivazione delle scelte per i permessi configurati

Le configurazioni scelte per questa esercitazione non sono casuali: ciascuna rappresenta uno scenario realistico con precise implicazioni di sicurezza.

La prima modifica applicata, `chmod 700` su `file.txt`, corrisponde alla configurazione più restrittiva possibile per un file con esecuzione abilitata. Essa è tipicamente adottata per script o file di configurazione contenenti credenziali o dati sensibili, in cui è fondamentale che nessun altro utente possa leggerne il contenuto.

La scelta di assegnare anche il permesso di esecuzione al proprietario riflette un caso d'uso frequente: file eseguibili o script di avvio che devono restare privati.

L'applicazione di `chmod 777` alla directory `test_permessi` ha perseguito un obiettivo didattico: rendere accessibile la directory a tutti gli utenti per poter verificare separatamente l'effetto dei permessi del file contenuto.

In un contesto produttivo, il valore `777` su una directory è generalmente sconsigliato perchè consente a qualsiasi utente di creare, rinominare ed eliminare file al suo interno, indipendentemente dalla proprietà dei singoli file.

Il test con `chmod 444` ha illustrato il comportamento di un file in sola lettura per tutti.

Questa configurazione è comune per file di documentazione, licenze o log immutabili, dove si vuole garantire che nessuno, neanche il proprietario senza un cambio esplicito di permessi, possa modificare il contenuto per errore.

Infine, `chmod 744` rappresenta un compromesso equilibrato tra accessibilità e controllo: il proprietario ha pieno controllo, mentre gli altri utenti possono solo leggere. È una configurazione adatta a script o binari distribuiti in ambienti condivisi, dove si vuole che gli utenti possano eseguirli o leggerli ma non modificarli.

5.2 Analisi dei risultati ottenuti durante i test

I risultati osservati durante i test hanno confermato in modo coerente il modello teorico dei permessi Unix.

Il caso più significativo è stato il tentativo dell'utente `kali` di modificare i permessi della directory `test_permessi`, di proprietà di `root`. Il sistema ha restituito il messaggio "Operation not permitted", dimostrando che in Linux solo il proprietario di un file (o l'utente `root`) può alterarne i permessi tramite `chmod`.

Questo principio impedisce escalation di privilegi non autorizzate e rappresenta una delle garanzie fondamentali del modello di sicurezza del kernel Linux.

Il test di lettura con `chmod 700` su `file.txt` ha confermato che anche avere accesso completo alla directory contenitrice (`test_permessi` era `777`) non garantisce la lettura di un file al suo interno se i permessi del file stesso la vietano. Questo evidenzia un aspetto importante: directory e file hanno permessi indipendenti, e l'accesso a un file richiede che entrambi i livelli siano adeguatamente configurati.

I test con `chmod 444` e `chmod 744`, invece, hanno verificato che i permessi "other" (il terzo gruppo di tre bit) si applicano correttamente all'utente `kali`, il quale non è né il proprietario del file né membro del gruppo `root`.

In entrambe le configurazioni il bit `r` era impostato per "others", e la lettura ha avuto successo.

Con `chmod 700` tale bit è stato azzerato, e l'accesso è stato correttamente negato.

Nota: Il comportamento del sistema è risultato perfettamente prevedibile in ogni scenario testato, confermando che la comprensione del modello ottale (tre cifre, ognuna da 0 a 7, per user/group/others) è sufficiente per pianificare con precisione la politica di accesso a qualsiasi risorsa del filesystem Linux.

Di seguito il riepilogo del sistema di permessi ottale applicato nell'esercitazione:

Configurazione	Ottale	Proprietario	Gruppo
Solo lettura (tutti)	444	r--	r--
Solo proprietario	700	rwX	---
Lettura proprietario+gruppo	744	rwX	r--
Accesso totale	777	rwX	rwX

Tabella 2 - Riepilogo delle configurazioni di permesso utilizzate durante l'esercitazione

6. Conclusioni

L'esercitazione mi ha permesso di consolidare la comprensione pratica del sistema di permessi di Linux, uno degli argomenti centrali nell'amministrazione sicura di sistemi Unix-like.

Attraverso la creazione di una directory e di un file, la modifica dei relativi permessi e i test di accesso da un utente non privilegiato, abbiamo potuto osservare direttamente come il kernel interpreta e applica le politiche di controllo degli accessi.

Un aspetto particolarmente utile è stato verificare che i permessi operano a due livelli distinti e indipendenti: quello della directory contenitrice e quello del file. Avere accesso alla directory non garantisce automaticamente l'accesso ai file in essa contenuti, e viceversa.

Questa granularità permette una gestione molto precisa delle autorizzazioni in ambienti multi-utente.

Abbiamo inoltre compreso il ruolo critico dell'utente root: solo il proprietario o il superutente possono modificare i permessi di una risorsa, rendendo il sistema resistente a tentativi di escalation di privilegi da parte di utenti non autorizzati. Questo principio, conosciuto come "principle of least privilege", è alla base di molte best practice di sicurezza in ambito sistemistico e, più in generale, in tutto il campo della cybersecurity.

La padronanza del comando `chmod` e della notazione ottale si conferma quindi una competenza fondamentale per chiunque voglia operare in modo sicuro su sistemi Linux, sia in ambito di amministrazione di sistema sia in contesti di penetration testing e hardening.